```
ai_controller.cpp
-----------------
// Copyright (c) 2022-2023 the Dviglo project
// Copyright (c) 2008-2023 the Urho3D project
// License: MIT

#include "ai_controller.h"

#include "ninja.h"

namespace Urho3D
{

static constexpr float INITIAL_AGGRESSION = 0.0020f;
static constexpr float INITIAL_PREDICTION = 30.f;
static constexpr float INITIAL_AIM_SPEED = 10.f;
static constexpr float DELTA_AGGRESSION = 0.000025f;
static constexpr float DELTA_PREDICTION = -0.15f;
static constexpr float DELTA_AIM_SPEED = 0.30f;
static constexpr float MAX_AGGRESSION = 0.01f;
static constexpr float MAX_PREDICTION = 20.f;
static constexpr float MAX_AIM_SPEED = 40.f;

float aiAggression = INITIAL_AGGRESSION;
float aiPrediction = INITIAL_PREDICTION;
float aiAimSpeed = INITIAL_AIM_SPEED;

void ResetAI()
{
    aiAggression = INITIAL_AGGRESSION;
    aiPrediction = INITIAL_PREDICTION;
    aiAimSpeed = INITIAL_AIM_SPEED;
}

void MakeAIHarder()
{
    aiAggression += DELTA_AGGRESSION;
    if (aiAggression > MAX_AGGRESSION)
        aiAggression = MAX_AGGRESSION;

    aiPrediction += DELTA_PREDICTION;
    if (aiPrediction < MAX_PREDICTION)
        aiPrediction = MAX_PREDICTION;

    aiAimSpeed += DELTA_AIM_SPEED;
    if (aiAimSpeed > MAX_AIM_SPEED)
        aiAimSpeed = MAX_AIM_SPEED;
}

void AIController::Control(Ninja* ownNinja, Node* ownNode, float timeStep)
{
    // Get new target if none. Do not constantly scan for new targets to conserve CPU time
    if (!currentTarget)
    {
        newTargetTimer += timeStep;
        if (newTargetTimer > 0.5)
            GetNewTarget(ownNode);
    }

    Node* targetNode = currentTarget.Get();

    if (targetNode)
    {
        // Check that current target is still alive. Otherwise choose new
        Ninja* targetNinja = targetNode->GetComponent<Ninja>();
        if (!targetNinja || targetNinja->health <= 0.f)
        {
            currentTarget = nullptr;
            return;
```

```
        }

        RigidBody* targetBody = targetNode->GetComponent<RigidBody>();

        ownNinja->controls.Set(CTRL_FIRE, false);
        ownNinja->controls.Set(CTRL_JUMP, false);

        float deltaX = 0.0f;
        float deltaY = 0.0f;

        // Aim from own head to target's feet
        Vector3 ownPos(ownNode->GetPosition() + Vector3(0.f, 0.9f, 0.f));
        Vector3 targetPos(targetNode->GetPosition() + Vector3(0.f, -0.9f, 0.f));
        float distance = (targetPos - ownPos).Length();

        // Use prediction according to target distance & estimated snowball speed
        Vector3 currentAim(ownNinja->GetAim() * Vector3(0.f, 0.f, 1.f));
        float predictDistance = distance;
        if (predictDistance > 50.f)
            predictDistance = 50.f;
        Vector3 predictedPos = targetPos + targetBody->GetLinearVelocity() * predictDistance /
aiPrediction;
        Vector3 targetAim = predictedPos - ownPos;

        // Add distance/height compensation
        float compensation = Max(targetAim.Length() - 15.f, 0.0f);
        targetAim += Vector3(0.f, 0.6f, 0.f) * compensation;

        // X-aiming
        targetAim.Normalize();
        Vector3 currentYaw(currentAim.x_, 0.f, currentAim.z_);
        Vector3 targetYaw(targetAim.x_, 0.f, targetAim.z_);
        currentYaw.Normalize();
        targetYaw.Normalize();
        deltaX = Clamp(Quaternion(currentYaw, targetYaw).YawAngle(), -aiAimSpeed, aiAimSpeed);

        // Y-aiming
        Vector3 currentPitch(0.f, currentAim.y_, 1.f);
        Vector3 targetPitch(0.f, targetAim.y_, 1.f);
        currentPitch.Normalize();
        targetPitch.Normalize();
        deltaY = Clamp(Quaternion(currentPitch, targetPitch).PitchAngle(), -aiAimSpeed,
aiAimSpeed);

        ownNinja->controls.yaw_ += 0.1f * deltaX;
        ownNinja->controls.pitch_ += 0.1f * deltaY;

        // Firing? if close enough and relatively correct aim
        if (distance < 25.f && currentAim.DotProduct(targetAim) > 0.75f)
        {
            if (Random(1.0f) < aiAggression)
                ownNinja->controls.Set(CTRL_FIRE, true);
        }

        // Movement
        ownNinja->dirChangeTime -= timeStep;
        if (ownNinja->dirChangeTime <= 0.f)
        {
            ownNinja->dirChangeTime = 0.5f + Random(1.0f);
            ownNinja->controls.Set(CTRL_UP | CTRL_DOWN | CTRL_LEFT | CTRL_RIGHT, false);

            // Far distance: go forward
            if (distance > 30.f)
                ownNinja->controls.Set(CTRL_UP, true);
            else if (distance > 6)
            {
                // Medium distance: random strafing, predominantly forward
                float v = Random(1.0f);
                if (v < 0.8f)
```

```
                    ownNinja->controls.Set(CTRL_UP, true);
                float h = Random(1.0f);
                if (h < 0.3f)
                    ownNinja->controls.Set(CTRL_LEFT, true);
                if (h > 0.7f)
                    ownNinja->controls.Set(CTRL_RIGHT, true);
            }
            else
            {
                // Close distance: random strafing backwards
                float v = Random(1.0f);
                if (v < 0.8f)
                    ownNinja->controls.Set(CTRL_DOWN, true);
                float h = Random(1.0f);
                if (h < 0.4f)
                    ownNinja->controls.Set(CTRL_LEFT, true);
                if (h > 0.6f)
                    ownNinja->controls.Set(CTRL_RIGHT, true);
            }
        }

        // Random jump, if going forward
        if (ownNinja->controls.IsDown(CTRL_UP) && distance < 1000.f)
        {
            if (Random(1.0f) < aiAggression / 5.0f)
                ownNinja->controls.Set(CTRL_JUMP, true);
        }
    }
    else
    {
        // If no target, walk idly
        ownNinja->controls.Set(CTRL_ALL, false);
        ownNinja->controls.Set(CTRL_UP, true);
        ownNinja->dirChangeTime -= timeStep;
        if (ownNinja->dirChangeTime <= 0.f)
        {
            ownNinja->dirChangeTime = 1.f + Random(2.f);
            ownNinja->controls.yaw_ += 0.1f * (Random(600.f) - 300.f);
        }
        if (ownNinja->isSliding)
            ownNinja->controls.yaw_ += 0.2f;
    }
}

void AIController::GetNewTarget(Node* ownNode)
{
    newTargetTimer = 0;

    Vector<Node*> nodes = ownNode->GetScene()->GetChildrenWithComponent("Ninja", true);
    float closestDistance = M_INFINITY;
    for (i32 i = 0; i < nodes.Size(); ++i)
    {
        Node* otherNode = nodes[i];
        Ninja* otherNinja = otherNode->GetComponent<Ninja>();
        if (otherNinja->side == SIDE_PLAYER && otherNinja->health > 0.f)
        {
            float distance = (ownNode->GetPosition() - otherNode->GetPosition()).LengthSquared();
            if (distance < closestDistance)
            {
                currentTarget = otherNode;
                closestDistance = distance;
            }
        }
    }
}

} // namespace Urho3D

================================================================
```

ai_controller.h

```
// Copyright (c) 2022-2023 the Dviglo project
// Copyright (c) 2008-2023 the Urho3D project
// License: MIT

#pragma once

#include <Urho3D/Urho3DAll.h>

namespace Urho3D
{

class Ninja;

void ResetAI();
void MakeAIHarder();

class AIController
{
private:
    // Use a weak handle instead of a normal handle to point to the current target
    // so that we don't mistakenly keep it alive.
    WeakPtr<Node> currentTarget;
    float newTargetTimer = 0;

public:
    void Control(Ninja* ownNinja, Node* ownNode, float timeStep);
    void GetNewTarget(Node* ownNode);
};

} // namespace Urho3D
```

==========
CMakeLists.txt

```
# Copyright (c) 2022-2023 the Dviglo project
# Copyright (c) 2008-2023 the Urho3D project
# License: MIT

if (NOT URHO3D_NETWORK OR NOT URHO3D_PHYSICS)
    return ()
endif ()

# Define target name
set (TARGET_NAME 80_ninja_snow_war)

# Define source files
define_source_files (GROUP RECURSE)

# Setup target with resource copying
setup_main_executable ()

# Setup test cases
setup_test ()
```

==========
foot_steps.cpp

```
// Copyright (c) 2022-2023 the Dviglo project
// Copyright (c) 2008-2023 the Urho3D project
// License: MIT

#include "foot_steps.h"
#include "utilities/spawn.h"

namespace Urho3D
{
```

```
void FootSteps::RegisterObject(Context* context)
{
    context->RegisterFactory<FootSteps>();
}

FootSteps::FootSteps(Context* context)
    : LogicComponent(context)
{
}

void FootSteps::Start()
{
    // Subscribe to animation triggers, which are sent by the AnimatedModel's node (same as our
node)
    SubscribeToEvent(node_, E_ANIMATIONTRIGGER, URHO3D_HANDLER(FootSteps,
HandleAnimationTrigger));
}

void FootSteps::HandleAnimationTrigger(StringHash eventType, VariantMap& eventData)
{
    using namespace AnimationTrigger;

    AnimatedModel* model = node_->GetComponent<AnimatedModel>();
    AnimationState* state = model->GetAnimationState(eventData[P_NAME].GetString());

    if (!state)
        return;

    // If the animation is blended with sufficient weight, instantiate a local particle effect for
the footstep.
    // The trigger data (string) tells the bone scenenode to use. Note: called on both client and
server
    if (state->GetWeight() > 0.5f)
    {
        Node* bone = node_->GetChild(eventData[P_DATA].GetString(), true);
        if (bone)
            SpawnParticleEffect(bone->GetScene(), bone->GetWorldPosition(),
"Particle/SnowExplosionFade.xml", 1, LOCAL);
    }
}

} // namespace Urho3D

================================================================================
foot_steps.h
------------
// Copyright (c) 2022-2023 the Dviglo project
// Copyright (c) 2008-2023 the Urho3D project
// License: MIT

#pragma once

#include <Urho3D/Urho3DAll.h>

namespace Urho3D
{

class FootSteps : public LogicComponent
{
    URHO3D_OBJECT(FootSteps, LogicComponent);

public:
    static void RegisterObject(Context* context);

    FootSteps(Context* context);

    void Start() override;
    void HandleAnimationTrigger(StringHash eventType, VariantMap& eventData);
};
```

```
} // namespace Urho3D
================================================================================
game_object.cpp
---------------
// Copyright (c) 2022-2023 the Dviglo project
// Copyright (c) 2008-2023 the Urho3D project
// License: MIT

#include "game_object.h"

namespace Urho3D
{

void GameObject::RegisterObject(Context* context)
{
    context->RegisterFactory<GameObject>();
}

GameObject::GameObject(Context* context)
    : LogicComponent(context)
    , onGround(false)
    , isSliding(false)
    , duration(-1) // Бесконечно
    , health(0)
    , maxHealth(0)
    , side(SIDE_NEUTRAL)
    , lastDamageSide(SIDE_NEUTRAL)
    , lastDamageCreatorID(0)
    , creatorID(0)
{
    SetUpdateEventMask(LogicComponentEvents::FixedUpdate);

    // if (runClient)
    //     Print("Warning! Logic object created on client!");
}

void GameObject::FixedUpdate(float timeStep)
{
    // Disappear when duration expired
    if (duration >= 0)
    {
        duration -= timeStep;
        if (duration <= 0)
            node_->Remove();
    }
}

bool GameObject::Damage(GameObject& origin, i32 amount)
{
    if (origin.side == side || health == 0)
        return false;

    lastDamageSide = origin.side;
    lastDamageCreatorID = origin.creatorID;
    health -= amount;
    if (health < 0)
        health = 0;
    return true;
}

bool GameObject::Heal(i32 amount)
{
    // By default do not heal
    return false;
}

void GameObject::PlaySound(const String& soundName)
{
```

```
    SoundSource3D* source = node_->CreateComponent<SoundSource3D>();
    Sound* sound = GetSubsystem<ResourceCache>()->GetResource<Sound>(soundName);
    // Subscribe to sound finished for cleaning up the source
    SubscribeToEvent(node_, E_SOUNDFINISHED, URHO3D_HANDLER(GameObject, HandleSoundFinished));

    source->SetDistanceAttenuation(2, 50, 1);
    source->Play(sound);
}

void GameObject::HandleSoundFinished(StringHash eventType, VariantMap& eventData)
{
    using namespace SoundFinished;
    SoundSource3D* source = static_cast<SoundSource3D*>(eventData[P_SOUNDSOURCE].GetPtr());
    source->Remove();
}

void GameObject::HandleNodeCollision(StringHash eventType, VariantMap& eventData)
{
    using namespace NodeCollision;
    Node* otherNode = static_cast<Node*>(eventData[P_OTHERNODE].GetPtr());
    RigidBody* otherBody = static_cast<RigidBody*>(eventData[P_OTHERBODY].GetPtr());

    // If the other collision shape belongs to static geometry, perform world collision
    if (otherBody->GetCollisionLayer() == 2)
        WorldCollision(eventData);

    // If the other node is scripted, perform object-to-object collision
    GameObject* otherObject = otherNode->GetDerivedComponent<GameObject>();
    if (otherObject)
        ObjectCollision(*otherObject, eventData);
}

void GameObject::WorldCollision(VariantMap& eventData)
{
    using namespace NodeCollision;
    MemoryBuffer contacts(eventData[P_CONTACTS].GetBuffer());

    while (!contacts.IsEof())
    {
        Vector3 contactPosition = contacts.ReadVector3();
        Vector3 contactNormal = contacts.ReadVector3();
        float contactDistance = contacts.ReadFloat();
        float contactImpulse = contacts.ReadFloat();

        // If contact is below node center and pointing up, assume it's ground contact
        if (contactPosition.y_ < node_->GetPosition().y_)
        {
            float level = contactNormal.y_;

            if (level > 0.75)
            {
                onGround = true;
            }
            else
            {
                // If contact is somewhere between vertical/horizontal, is sliding a slope
                if (level > 0.1)
                    isSliding = true;
            }
        }
    }

    // Ground contact has priority over sliding contact
    if (onGround == true)
        isSliding = false;
}

void GameObject::ObjectCollision(GameObject& otherObject, VariantMap& eventData)
{
```

```
}

void GameObject::ResetWorldCollision()
{
    RigidBody* body = node_->GetComponent<RigidBody>();

    if (body->IsActive())
    {
        onGround = false;
        isSliding = false;
    }
    else
    {
        // If body is not active, assume it rests on the ground
        onGround = true;
        isSliding = false;
    }
}

} // namespace Urho3D

================================================================================
game_object.h
---------------
// Copyright (c) 2022-2023 the Dviglo project
// Copyright (c) 2008-2023 the Urho3D project
// License: MIT

#pragma once

#include <Urho3D/Urho3DAll.h>

namespace Urho3D
{

constexpr i32 CTRL_UP = 1;
constexpr i32 CTRL_DOWN = 2;
constexpr i32 CTRL_LEFT = 4;
constexpr i32 CTRL_RIGHT = 8;
constexpr i32 CTRL_FIRE = 16;
constexpr i32 CTRL_JUMP = 32;
constexpr i32 CTRL_ALL = 63;

constexpr i32 SIDE_NEUTRAL = 0;
constexpr i32 SIDE_PLAYER = 1;
constexpr i32 SIDE_ENEMY = 2;

class GameObject : public LogicComponent
{
    URHO3D_OBJECT(GameObject, LogicComponent);

public:
    static void RegisterObject(Context* context);

    bool onGround;
    bool isSliding;
    float duration;
    i32 health;
    i32 maxHealth;
    i32 side; // Какой стороне принадлежит (игроки, враги или нейтралы)
    i32 lastDamageSide;
    i32 lastDamageCreatorID;
    i32 creatorID;

    GameObject(Context* context);

    void FixedUpdate(float timeStep) override;
    bool Damage(GameObject& origin, i32 amount);
    virtual bool Heal(i32 amount);
```

```
    void PlaySound(const String& soundName);
    void HandleSoundFinished(StringHash eventType, VariantMap& eventData);
    void HandleNodeCollision(StringHash eventType, VariantMap& eventData);
    virtual void WorldCollision(VariantMap& eventData);
    virtual void ObjectCollision(GameObject& otherObject, VariantMap& eventData);
    void ResetWorldCollision();
};

} // namespace Urho3D

================================================================================
light_flash.cpp
---------------
// Copyright (c) 2022-2023 the Dviglo project
// Copyright (c) 2008-2023 the Urho3D project
// License: MIT

#include "light_flash.h"

namespace Urho3D
{

void LightFlash::RegisterObject(Context* context)
{
    context->RegisterFactory<LightFlash>();
}

LightFlash::LightFlash(Context* context)
    : GameObject(context)
{
    duration = 2.0f;
}

void LightFlash::FixedUpdate(float timeStep)
{
    Light* light = node_->GetComponent<Light>();
    light->SetBrightness(light->GetBrightness() * Max(1.0f - timeStep * 10.0f, 0.0f));

    // Call superclass to handle lifetime
    GameObject::FixedUpdate(timeStep);
}

} // namespace Urho3D

================================================================================
light_flash.h
-------------
// Copyright (c) 2022-2023 the Dviglo project
// Copyright (c) 2008-2023 the Urho3D project
// License: MIT

#pragma once

#include "game_object.h"

namespace Urho3D
{

class LightFlash : public GameObject
{
    URHO3D_OBJECT(LightFlash, GameObject);

public:
    static void RegisterObject(Context* context);

    LightFlash(Context* context);
    void FixedUpdate(float timeStep) override;
};
```

```
} // namespace Urho3D

================================================================================
main.cpp
-----------
// Copyright (c) 2022-2023 the Dviglo project
// Copyright (c) 2008-2023 the Urho3D project
// License: MIT

/*
    Простой запуск игры - одиночная игра.
    При запуске с параметром "-nobgm" фоновая музыка будет отключена.
    С параметром "-server" игра запустится в режиме сервера.
    Сервер удобно запускать в виде консольного приложения, используя параметры "-server -
headless".
    С параметром "-address 127.0.0.1" игра запустится в режиме клиента и
    подключится к серверу, запущенному на этом же компьютере.
    Также игра поддерживает геймпады и сенсорные экраны.
    Параметр "-touch" позволяет сэмулировать тачскрин на ПК.
    Друг другу игроки урон не наносят.
*/

#include <Urho3D/Urho3DAll.h>

#include "foot_steps.h"
#include "light_flash.h"
#include "ninja.h"
#include "player.h"
#include "potion.h"
#include "snow_crate.h"
#include "snowball.h"
#include "utilities/network.h"
#include "utilities/spawn.h"

namespace Urho3D
{

static constexpr float MOUSE_SENSITIVITY = 0.125f;
static constexpr float TOUCH_SENSITIVITY = 2.0f;
static constexpr float JOY_SENSITIVITY = 0.5f;
static constexpr float JOY_MOVE_DEAD_ZONE = 0.333f;
static constexpr float JOY_LOOK_DEAD_ZONE = 0.05f;
static constexpr float CAMERA_MIN_DIST = 0.25f;
static constexpr float CAMERA_MAX_DIST = 5.0f;
static constexpr float CAMERA_SAFETY_DIST = 0.3f;
static constexpr i32 INITIAL_MAX_ENEMIES = 5;
static constexpr i32 FINAL_MAX_ENEMIES = 25;
static constexpr i32 MAX_POWERUPS = 5;
static constexpr i32 INCREMENT_EACH = 10;
static constexpr i32 PLAYER_HEALTH = 20;
static constexpr float ENEMY_SPAWN_RATE = 1.0f;
static constexpr float POWERUP_SPAWN_RATE = 15.0f;
static constexpr float SPAWN_AREA_SIZE = 5.0f;

class App : public Application
{
    URHO3D_OBJECT(App, Application);

private:
    SharedPtr<Scene> gameScene;
    SharedPtr<Node> gameCameraNode;
    Camera* gameCamera = nullptr;
    SharedPtr<Node> musicNode;
    SoundSource* musicSource = nullptr;
    SharedPtr<Text> scoreText;
    SharedPtr<Text> hiscoreText;
    SharedPtr<Text> messageText;
    SharedPtr<BorderImage> healthBar;
    SharedPtr<BorderImage> sight;
```

```
    Controls playerControls;
    Controls prevPlayerControls;
    bool singlePlayer = true;
    bool gameOn = false;
    bool drawDebug = false;
    bool drawOctreeDebug = false;
    i32 maxEnemies = 0;
    i32 incrementCounter = 0;
    float enemySpawnTimer = 0.f;
    float powerupSpawnTimer = 0.f;
    NodeId clientNodeID = 0;
    i32 clientScore = 0;

    i32 screenJoystickID = -1;
    i32 screenJoystickSettingsID = -1;
    bool touchEnabled = false;

    Vector<Player> players;
    Vector<HiscoreEntry> hiscores;

public:
    App(Context* context)
        : Application(context)
    {
        Ninja::RegisterObject(context);
        FootSteps::RegisterObject(context);
        GameObject::RegisterObject(context);
        LightFlash::RegisterObject(context);
        Potion::RegisterObject(context);
        SnowCrate::RegisterObject(context);
        Snowball::RegisterObject(context);
    }

    void Setup() override
    {
        // Modify engine startup parameters
        engineParameters_[EP_WINDOW_TITLE] = "Ninja Snow Wars Native";
        engineParameters_[EP_LOG_NAME] = GetSubsystem<FileSystem>()-
>GetAppPreferencesDir("urho3d", "logs") + "80_ninja_snow_wars_native.log";
        //engineParameters_[EP_FULL_SCREEN] = false;
    }

    void Start() override
    {
        if (GetSubsystem<Engine>()->IsHeadless())
            OpenConsoleWindow();

        ParseNetworkArguments();
        if (runServer || runClient)
            singlePlayer = false;

        InitAudio();
        InitConsole();
        InitScene();
        InitNetworking();
        CreateCamera();
        CreateOverlays();

        SubscribeToEvent(gameScene, E_SCENEUPDATE, URHO3D_HANDLER(App, HandleUpdate));

        PhysicsWorld* physicsWorld = gameScene->GetComponent<PhysicsWorld>();
        if (physicsWorld)
            SubscribeToEvent(physicsWorld, E_PHYSICSPRESTEP, URHO3D_HANDLER(App,
HandleFixedUpdate));

        SubscribeToEvent(gameScene, E_SCENEPOSTUPDATE, URHO3D_HANDLER(App, HandlePostUpdate));
        SubscribeToEvent(E_POSTRENDERUPDATE, URHO3D_HANDLER(App, HandlePostRenderUpdate));
        SubscribeToEvent(E_KEYDOWN, URHO3D_HANDLER(App, HandleKeyDown));
```

```
        SubscribeToEvent("Points", URHO3D_HANDLER(App, HandlePoints));
        SubscribeToEvent("Kill", URHO3D_HANDLER(App, HandleKill));
        SubscribeToEvent(E_SCREENMODE, URHO3D_HANDLER(App, HandleScreenMode));

        if (singlePlayer)
        {
            StartGame(nullptr);
            GetSubsystem<Engine>()->SetPauseMinimized(true);
        }
    }

    void InitAudio()
    {
        if (GetSubsystem<Engine>()->IsHeadless())
            return;

        Audio* audio = GetSubsystem<Audio>();

        // Lower mastervolumes slightly.
        audio->SetMasterGain(SOUND_MASTER, 0.75f);
        audio->SetMasterGain(SOUND_MUSIC, 0.9f);

        if (!nobgm)
        {
            Sound* musicFile = GetSubsystem<ResourceCache>()->GetResource<Sound>("Music/Ninja
Gods.ogg");
            musicFile->SetLooped(true);

            // Note: the non-positional sound source component need to be attached to a node to
become effective
            // Due to networked mode clearing the scene on connect, do not attach to the scene
itself
            musicNode = new Node(context_);
            musicSource = musicNode->CreateComponent<SoundSource>();
            musicSource->SetSoundType(SOUND_MUSIC);
            musicSource->Play(musicFile);
        }
    }

    void InitConsole()
    {
        Engine* engine = GetSubsystem<Engine>();

        if (engine->IsHeadless())
            return;

        XMLFile* uiStyle = GetSubsystem<ResourceCache>()->GetResource<XMLFile>
("UI/DefaultStyle.xml");
        GetSubsystem<UI>()->GetRoot()->SetDefaultStyle(uiStyle);

        Console* console = engine->CreateConsole();
        console->SetDefaultStyle(uiStyle);
        console->GetBackground()->SetOpacity(0.8f);

        DebugHud* debug_hud = engine->CreateDebugHud();
        debug_hud->SetDefaultStyle(uiStyle);
    }

    void InitScene()
    {
        gameScene = new Scene(context_);
        gameScene->SetName("NinjaSnowWar");

        // For the multiplayer client, do not load the scene, let it load from the server
        if (runClient)
            return;

        ResourceCache* cache = GetSubsystem<ResourceCache>();
        Renderer* renderer = GetSubsystem<Renderer>();
```

```
        Engine* engine = GetSubsystem<Engine>();
        Graphics* graphics = GetSubsystem<Graphics>();

        gameScene->LoadXML(*cache->GetFile("Scenes/NinjaSnowWar.xml"));

        // On mobile devices render the shadowmap first for better performance, adjust the
cascaded shadows
        String platform = GetPlatform();
        if (platform == "Android" || platform == "iOS" || platform == "Raspberry Pi")
        {
            renderer->SetReuseShadowMaps(false);
            // Adjust the directional light shadow range slightly further, as only the first
            // cascade is supported
            Node* dirLightNode = gameScene->GetChild("GlobalLight", true);
            if (dirLightNode)
            {
                Light* dirLight = dirLightNode->GetComponent<Light>();
                dirLight->SetShadowCascade(CascadeParameters(15.0f, 0.0f, 0.0f, 0.0f, 0.9f));
            }
        }

        // Precache shaders if possible
        if (!engine->IsHeadless() && cache->Exists("NinjaSnowWarShaders.xml"))
            graphics->PrecacheShaders(*cache->GetFile("NinjaSnowWarShaders.xml"));
    }

    void InitNetworking()
    {
        Network* network = GetSubsystem<Network>();

        network->SetUpdateFps(25); // 1/4 of physics FPS
        // Remote events sent between client & server must be explicitly registered or else they
are not allowed to be received
        network->RegisterRemoteEvent("PlayerSpawned");
        network->RegisterRemoteEvent("UpdateScore");
        network->RegisterRemoteEvent("UpdateHiscores");
        network->RegisterRemoteEvent("ParticleEffect");

        if (runServer)
        {
            network->StartServer(serverPort);

            // Disable physics interpolation to ensure clients get sent physically correct
transforms
            gameScene->GetComponent<PhysicsWorld>()->SetInterpolation(false);

            SubscribeToEvent(E_CLIENTIDENTITY, URHO3D_HANDLER(App, HandleClientIdentity));
            SubscribeToEvent(E_CLIENTSCENELOADED, URHO3D_HANDLER(App, HandleClientSceneLoaded));
            SubscribeToEvent(E_CLIENTDISCONNECTED, URHO3D_HANDLER(App, HandleClientDisconnected));
        }

        if (runClient)
        {
            VariantMap identity;
            identity["UserName"] = userName;
            network->SetUpdateFps(50); // Increase controls send rate for better responsiveness
            network->Connect(serverAddress, serverPort, gameScene, identity);

            SubscribeToEvent("PlayerSpawned", URHO3D_HANDLER(App, HandlePlayerSpawned));
            SubscribeToEvent("UpdateScore", URHO3D_HANDLER(App, HandleUpdateScore));
            SubscribeToEvent("UpdateHiscores", URHO3D_HANDLER(App, HandleUpdateHiscores));
            SubscribeToEvent(E_NETWORKUPDATESENT, URHO3D_HANDLER(App, HandleNetworkUpdateSent));
        }
    }

    void InitTouchInput()
    {
        Input* input = GetSubsystem<Input>();
        ResourceCache* cache = GetSubsystem<ResourceCache>();
```

```
        touchEnabled = true;
        screenJoystickID = input->AddScreenJoystick(cache->GetResource<XMLFile>
("UI/ScreenJoystick_NinjaSnowWar.xml"));
    }

    void CreateCamera()
    {
        // Note: the camera is not in the scene
        gameCameraNode = new Node(context_);
        gameCameraNode->SetPosition(Vector3(0.f, 2.f, -10.f));

        gameCamera = gameCameraNode->CreateComponent<Camera>();
        gameCamera->SetNearClip(0.5f);
        gameCamera->SetFarClip(160.f);

        Engine* engine = GetSubsystem<Engine>();
        Renderer* renderer = GetSubsystem<Renderer>();
        Audio* audio = GetSubsystem<Audio>();

        if (!engine->IsHeadless())
        {
            SharedPtr<Viewport> viewport(new Viewport(context_, gameScene, gameCamera));
            renderer->SetViewport(0, viewport);
            audio->SetListener(gameCameraNode->CreateComponent<SoundListener>());
        }
    }

    void CreateOverlays()
    {
        if (GetSubsystem<Engine>()->IsHeadless() || runServer)
            return;

        i32 height = GetSubsystem<Graphics>()->GetHeight() / 22;
        if (height > 64)
            height = 64;

        ResourceCache* cache = GetSubsystem<ResourceCache>();
        UI* ui = GetSubsystem<UI>();
        Input* input = GetSubsystem<Input>();

        sight = new BorderImage(context_);
        sight->SetTexture(cache->GetResource<Texture2D>("Textures/NinjaSnowWar/Sight.png"));
        sight->SetAlignment(HA_CENTER, VA_CENTER);
        sight->SetSize(height, height);
        ui->GetRoot()->AddChild(sight);

        Font* font = cache->GetResource<Font>("Fonts/BlueHighway.ttf");

        scoreText = new Text(context_);
        scoreText->SetFont(font, 13.f);
        scoreText->SetAlignment(HA_LEFT, VA_TOP);
        scoreText->SetPosition(5, 5);
        scoreText->SetColor(C_BOTTOMLEFT, Color(1.f, 1.f, 0.25f));
        scoreText->SetColor(C_BOTTOMRIGHT, Color(1.f, 1.f, 0.25f));
        ui->GetRoot()->AddChild(scoreText);

        hiscoreText = new Text(context_);
        hiscoreText->SetFont(font, 13.f);
        hiscoreText->SetAlignment(HA_RIGHT, VA_TOP);
        hiscoreText->SetPosition(-5, 5);
        hiscoreText->SetColor(C_BOTTOMLEFT, Color(1.f, 1.f, 0.25f));
        hiscoreText->SetColor(C_BOTTOMRIGHT, Color(1.f, 1.f, 0.25f));
        ui->GetRoot()->AddChild(hiscoreText);

        messageText = new Text(context_);
        messageText->SetFont(font, 13.f);
        messageText->SetAlignment(HA_CENTER, VA_CENTER);
        messageText->SetPosition(0, -height * 2);
```

```
        messageText->SetColor(Color(1.f, 0.f, 0.f));
        ui->GetRoot()->AddChild(messageText);

        SharedPtr<BorderImage> healthBorder(new BorderImage(context_));
        healthBorder->SetTexture(cache->GetResource<Texture2D>
("Textures/NinjaSnowWar/HealthBarBorder.png"));
        healthBorder->SetAlignment(HA_CENTER, VA_TOP);
        healthBorder->SetPosition(0, 8);
        healthBorder->SetSize(120, 20);
        ui->GetRoot()->AddChild(healthBorder);

        healthBar = new BorderImage(context_);
        healthBar->SetTexture(cache->GetResource<Texture2D>
("Textures/NinjaSnowWar/HealthBarInside.png"));
        healthBar->SetPosition(2, 2);
        healthBar->SetSize(116, 16);
        healthBorder->AddChild(healthBar);

        if (GetPlatform() == "Android" || GetPlatform() == "iOS")
            // On mobile platform, enable touch by adding a screen joystick
            InitTouchInput();
        else if (input->GetNumJoysticks() == 0)
            // On desktop platform, do not detect touch when we already got a joystick
            SubscribeToEvent(E_TOUCHBEGIN, URHO3D_HANDLER(App, HandleTouchBegin));
    }

    void SetMessage(const String& message)
    {
        if (messageText)
            messageText->SetText(message);
    }

    void StartGame(Connection* connection)
    {
        // Clear the scene of all existing scripted objects
        {
            Vector<Node*> scriptedNodes;

            for (Node* child : gameScene->GetChildren())
            {
                for (Component* component : child->GetComponents())
                {
                    // Проверяем, что компонент - производный от GameObject
                    GameObject* gameObject = dynamic_cast<GameObject*>(component);

                    if (gameObject)
                        scriptedNodes.Push(child);
                }
            }

            for (Node* node : scriptedNodes)
                node->Remove();
        }

        players.Clear();
        SpawnPlayer(connection);

        ResetAI();

        gameOn = true;
        maxEnemies = INITIAL_MAX_ENEMIES;
        incrementCounter = 0;
        enemySpawnTimer = 0;
        powerupSpawnTimer = 0;

        if (singlePlayer)
        {
            playerControls.yaw_ = 0;
            playerControls.pitch_ = 0;
```

```
            SetMessage("");
        }
    }

    void SpawnPlayer(Connection* connection)
    {
        Vector3 spawnPosition;
        if (singlePlayer)
            spawnPosition = Vector3(0.f, 0.97f, 0.f);
        else
            spawnPosition = Vector3(Random(SPAWN_AREA_SIZE) - SPAWN_AREA_SIZE * 0.5f, 0.97f,
Random(SPAWN_AREA_SIZE) - SPAWN_AREA_SIZE);

        Node* playerNode = SpawnObject(gameScene, spawnPosition, Quaternion(), "ninja");
        // Set owner connection. Owned nodes are always updated to the owner at full frequency
        playerNode->SetOwner(connection);
        playerNode->SetName("Player");

        // Initialize variables
        Ninja* playerNinja = playerNode->GetComponent<Ninja>();
        playerNinja->health = playerNinja->maxHealth = PLAYER_HEALTH;
        playerNinja->side = SIDE_PLAYER;
        // Make sure the player can not shoot on first frame by holding the button down
        if (!connection)
            playerNinja->controls = playerNinja->prevControls = playerControls;
        else
            playerNinja->controls = playerNinja->prevControls = connection->GetControls();

        // Check if player entry already exists
        i32 playerIndex = -1;
        for (i32 i = 0; i < players.Size(); ++i)
        {
            if (players[i].connection == connection)
            {
                playerIndex = i;
                break;
            }
        }

        // Does not exist, create new
        if (playerIndex < 0)
        {
            playerIndex = players.Size();
            players.Resize(players.Size() + 1);
            players[playerIndex].connection = connection;

            if (connection)
            {
                players[playerIndex].name = connection->identity_["UserName"].GetString();
                // In multiplayer, send current hiscores to the new player
                SendHiscores(playerIndex);
            }
            else
            {
                players[playerIndex].name = "Player";
                // In singleplayer, create also the default hiscore entry immediately
                HiscoreEntry newHiscore;
                newHiscore.name = players[playerIndex].name;
                newHiscore.score = 0;
                hiscores.Push(newHiscore);
            }
        }

        players[playerIndex].nodeID = playerNode->GetID();
        players[playerIndex].score = 0;

        if (connection)
        {
            // In multiplayer, send initial score, then send a remote event that tells the spawned
```

```
  node's ID
            // It is important for the event to be in-order so that the node has been replicated
  first
            SendScore(playerIndex);
            VariantMap eventData;
            eventData["NodeID"] = playerNode->GetID();
            connection->SendRemoteEvent("PlayerSpawned", true, eventData);

            // Create name tag (Text3D component) for players in multiplayer
            Node* textNode = playerNode->CreateChild("NameTag");
            textNode->SetPosition(Vector3(0.f, 1.2f, 0.f));
            Text3D* text3D = textNode->CreateComponent<Text3D>();
            Font* font = GetSubsystem<ResourceCache>()->GetResource<Font>
  ("Fonts/BlueHighway.ttf");
            text3D->SetFont(font, 19.f);
            text3D->SetColor(Color(1.f, 1.f, 0.f));
            text3D->SetText(players[playerIndex].name);
            text3D->SetHorizontalAlignment(HA_CENTER);
            text3D->SetVerticalAlignment(VA_CENTER);
            text3D->SetFaceCameraMode(FC_ROTATE_XYZ);
        }
    }

    void HandleUpdate(StringHash eventType, VariantMap& eventData)
    {
        float timeStep = eventData["TimeStep"].GetFloat();

        UpdateControls();
        CheckEndAndRestart();

        if (GetSubsystem<Engine>()->IsHeadless())
        {
            String command = GetConsoleInput();
            if (command.Length() > 0)
                GetSubsystem<Script>()->Execute(command);
        }
        else
        {
            DebugHud* debugHud = GetSubsystem<DebugHud>();

            if (debugHud->GetMode() != DebugHudElements::None)
            {
                Node* playerNode = FindOwnNode();
                if (playerNode)
                {
                    debugHud->SetAppStats("Player Pos", playerNode-
  >GetWorldPosition().ToString());
                    debugHud->SetAppStats("Player Yaw", Variant(playerNode-
  >GetWorldRotation().YawAngle()));
                }
                else
                {
                    debugHud->ClearAppStats();
                }
            }
        }
    }

    void HandleFixedUpdate(StringHash eventType, VariantMap& eventData)
    {
        float timeStep = eventData["TimeStep"].GetFloat();

        // Spawn new objects, singleplayer or server only
        if (singlePlayer || runServer)
            SpawnObjects(timeStep);
    }

    void HandlePostUpdate(StringHash eventType, VariantMap& eventData)
    {
```

```
        UpdateCamera();
        UpdateStatus();
    }

    void HandlePostRenderUpdate(StringHash eventType, VariantMap& eventData)
    {
        if (GetSubsystem<Engine>()->IsHeadless())
            return;

        if (drawDebug)
            gameScene->GetComponent<PhysicsWorld>()->DrawDebugGeometry(true);

        if (drawOctreeDebug)
            gameScene->GetComponent<Octree>()->DrawDebugGeometry(true);
    }

    void HandleTouchBegin(StringHash eventType, VariantMap& eventData)
    {
        // On some platforms like Windows the presence of touch input can only be detected
dynamically
        InitTouchInput();
        UnsubscribeFromEvent("TouchBegin");
    }

    void HandleKeyDown(StringHash eventType, VariantMap& eventData)
    {
        Console* console = GetSubsystem<Console>();
        DebugHud* debugHud = GetSubsystem<DebugHud>();
        Engine* engine = GetSubsystem<Engine>();
        Graphics* graphics = GetSubsystem<Graphics>();
        FileSystem* fileSystem = GetSubsystem<FileSystem>();
        Time* time = GetSubsystem<Time>();
        Audio* audio = GetSubsystem<Audio>();
        Input* input = GetSubsystem<Input>();
        ResourceCache* cache = GetSubsystem<ResourceCache>();

        i32 key = eventData["Key"].GetI32();

        if (key == KEY_ESCAPE)
        {
            if (!console->IsVisible())
                engine->Exit();
            else
                console->SetVisible(false);
        }

        else if (key == KEY_F1)
            console->Toggle();

        else if (key == KEY_F2)
            debugHud->ToggleAll();

        else if (key == KEY_F3)
            drawDebug = !drawDebug;

        else if (key == KEY_F4)
            drawOctreeDebug = !drawOctreeDebug;

        else if (key == KEY_F5)
            debugHud->Toggle(DebugHudElements::EventProfiler);

        // Take screenshot
        else if (key == KEY_F6)
        {
            Image screenshot(context_);
            graphics->TakeScreenShot(screenshot);
            // Here we save in the Data folder with date and time appended
            screenshot.SavePNG(fileSystem->GetProgramDir() + "Data/Screenshot_" +
                time->GetTimeStamp().Replaced(':', '_').Replaced('.', '_').Replaced(' ', '_') +
```

```
".png");
        }
        // Allow pause only in singleplayer
        if (key == KEY_P && singlePlayer && !console->IsVisible() && gameOn)
        {
            gameScene->SetUpdateEnabled(!gameScene->IsUpdateEnabled());
            if (!gameScene->IsUpdateEnabled())
            {
                SetMessage("PAUSED");
                audio->PauseSoundType(SOUND_EFFECT);

                // Open the settings joystick only if the controls screen joystick was already
open
                if (screenJoystickID >= 0)
                {
                    // Lazy initialization
                    if (screenJoystickSettingsID < 0)
                        screenJoystickSettingsID = input->AddScreenJoystick(cache-
>GetResource<XMLFile>("UI/ScreenJoystickSettings_NinjaSnowWar.xml"));
                    else
                        input->SetScreenJoystickVisible(screenJoystickSettingsID, true);
                }
            }
            else
            {
                SetMessage("");
                audio->ResumeSoundType(SOUND_EFFECT);

                // Hide the settings joystick
                if (screenJoystickSettingsID >= 0)
                    input->SetScreenJoystickVisible(screenJoystickSettingsID, false);
            }
        }
    }

    void HandlePoints(StringHash eventType, VariantMap& eventData)
    {
        if (eventData["DamageSide"].GetI32() == SIDE_PLAYER)
        {
            // Get node ID of the object that should receive points -> use it to find player index
            i32 playerIndex = FindPlayerIndex(eventData["Receiver"].GetI32());
            if (playerIndex >= 0)
            {
                players[playerIndex].score += eventData["Points"].GetI32();
                SendScore(playerIndex);

                bool newHiscore = CheckHiscore(playerIndex);
                if (newHiscore)
                    SendHiscores(-1);
            }
        }
    }

    void HandleKill(StringHash eventType, VariantMap& eventData)
    {
        if (eventData["DamageSide"].GetI32() == SIDE_PLAYER)
        {
            MakeAIHarder();

            // Increment amount of simultaneous enemies after enough kills
            incrementCounter++;
            if (incrementCounter >= INCREMENT_EACH)
            {
                incrementCounter = 0;
                if (maxEnemies < FINAL_MAX_ENEMIES)
                    maxEnemies++;
            }
        }
    }
```

```
    void HandleClientIdentity(StringHash eventType, VariantMap& eventData)
    {
        Connection* connection = (Connection*)GetEventSender();
        // If user has empty name, invent one
        if (connection->identity_["UserName"].GetString().Trimmed().Empty())
            connection->identity_["UserName"] = "user" + String(Random(1000));
        // Assign scene to begin replicating it to the client
        connection->SetScene(gameScene);
    }

    void HandleClientSceneLoaded(StringHash eventType, VariantMap& eventData)
    {
        // Now client is actually ready to begin. If first player, clear the scene and restart the
game
        Connection* connection = (Connection*)GetEventSender();
        if (players.Empty())
            StartGame(connection);
        else
            SpawnPlayer(connection);
    }

    void HandleClientDisconnected(StringHash eventType, VariantMap& eventData)
    {
        Connection* connection = (Connection*)GetEventSender();
        // Erase the player entry, and make the player's ninja commit seppuku (if exists)
        for (i32 i = 0; i < players.Size(); ++i)
        {
            if (players[i].connection == connection)
            {
                players[i].connection = nullptr;
                Node* playerNode = FindPlayerNode(i);
                if (playerNode)
                {
                    Ninja* playerNinja = playerNode->GetComponent<Ninja>();
                    playerNinja->health = 0;
                    playerNinja->lastDamageSide = SIDE_NEUTRAL; // No-one scores from this
                }
                players.Erase(i);
                return;
            }
        }
    }

    void HandlePlayerSpawned(StringHash eventType, VariantMap& eventData)
    {
        // Store our node ID and mark the game as started
        clientNodeID = eventData["NodeID"].GetI32();
        gameOn = true;
        SetMessage("");

        // Copy initial yaw from the player node (we should have it replicated now)
        Node* playerNode = FindOwnNode();
        if (playerNode)
        {
            playerControls.yaw_ = playerNode->GetRotation().YawAngle();
            playerControls.pitch_ = 0.f;

            // Disable the nametag from own character
            Node* nameTag = playerNode->GetChild("NameTag");
            nameTag->SetEnabled(false);
        }
    }

    void HandleUpdateScore(StringHash eventType, VariantMap& eventData)
    {
        clientScore = eventData["Score"].GetI32();
        scoreText->SetText("Score " + String(clientScore));
    }
```

```
void HandleUpdateHiscores(StringHash eventType, VariantMap& eventData)
{
    VectorBuffer data(eventData["Hiscores"].GetBuffer());
    hiscores.Resize(data.ReadVLE());
    for (i32 i = 0; i < hiscores.Size(); ++i)
    {
        hiscores[i].name = data.ReadString();
        hiscores[i].score = data.ReadI32();
    }

    String allHiscores;
    for (i32 i = 0; i < hiscores.Size(); ++i)
        allHiscores += hiscores[i].name + " " + String(hiscores[i].score) + "\n";
    hiscoreText->SetText(allHiscores);
}

void HandleNetworkUpdateSent(StringHash eventType, VariantMap& eventData)
{
    Network* network = GetSubsystem<Network>();
    // Clear accumulated buttons from the network controls
    if (network->GetServerConnection() != nullptr)
        network->GetServerConnection()->controls_.Set(CTRL_ALL, false);
}

i32 FindPlayerIndex(NodeId nodeID)
{
    for (i32 i = 0; i < players.Size(); ++i)
    {
        if (players[i].nodeID == nodeID)
            return i;
    }
    return -1;
}

Node* FindPlayerNode(i32 playerIndex)
{
    if (playerIndex >= 0 && playerIndex < players.Size())
        return gameScene->GetNode(players[playerIndex].nodeID);
    else
        return nullptr;
}

Node* FindOwnNode()
{
    if (singlePlayer)
        return gameScene->GetChild("Player", true);
    else
        return gameScene->GetNode(clientNodeID);
}

bool CheckHiscore(i32 playerIndex)
{
    for (i32 i = 0; i < hiscores.Size(); ++i)
    {
        if (hiscores[i].name == players[playerIndex].name)
        {
            if (players[playerIndex].score > hiscores[i].score)
            {
                hiscores[i].score = players[playerIndex].score;
                SortHiscores();
                return true;
            }
            else
                return false; // No update to individual hiscore
        }
    }

    // Not found, create new hiscore entry
```

```
        HiscoreEntry newHiscore;
        newHiscore.name = players[playerIndex].name;
        newHiscore.score = players[playerIndex].score;
        hiscores.Push(newHiscore);
        SortHiscores();
        return true;
    }

    void SortHiscores()
    {
        for (i32 i = 1; i < hiscores.Size(); ++i)
        {
            HiscoreEntry temp = hiscores[i];
            i32 j = i;
            while (j > 0 && temp.score > hiscores[j - 1].score)
            {
                hiscores[j] = hiscores[j - 1];
                --j;
            }
            hiscores[j] = temp;
        }
    }

    void SendScore(i32 playerIndex)
    {
        if (!runServer || playerIndex < 0 || playerIndex >= players.Size())
            return;

        VariantMap eventData;
        eventData["Score"] = players[playerIndex].score;
        players[playerIndex].connection->SendRemoteEvent("UpdateScore", true, eventData);
    }

    void SendHiscores(int playerIndex)
    {
        if (!runServer)
            return;

        VectorBuffer data;
        data.WriteVLE(hiscores.Size());
        for (i32 i = 0; i < hiscores.Size(); ++i)
        {
            data.WriteString(hiscores[i].name);
            data.WriteI32(hiscores[i].score);
        }

        VariantMap eventData;
        eventData["Hiscores"] = data;

        if (playerIndex >= 0 && playerIndex < players.Size())
            players[playerIndex].connection->SendRemoteEvent("UpdateHiscores", true, eventData);
        else
            GetSubsystem<Network>()->BroadcastRemoteEvent(gameScene, "UpdateHiscores", true,
eventData); // Send to all in scene
    }

    void SpawnObjects(float timeStep)
    {
        // If game not running, run only the random generator
        if (!gameOn)
        {
            Random();
            return;
        }

        // Spawn powerups
        powerupSpawnTimer += timeStep;
        if (powerupSpawnTimer >= POWERUP_SPAWN_RATE)
        {
```

```
            powerupSpawnTimer = 0;
            i32 numPowerups = gameScene->GetChildrenWithComponent("SnowCrate", true).Size() +
gameScene->GetChildrenWithComponent("Potion", true).Size();

            if (numPowerups < MAX_POWERUPS)
            {
                const float maxOffset = 40.f;
                float xOffset = Random(maxOffset * 2.0f) - maxOffset;
                float zOffset = Random(maxOffset * 2.0f) - maxOffset;

                SpawnObject(gameScene, Vector3(xOffset, 50.f, zOffset), Quaternion(),
"snow_crate");
            }
        }

        // Spawn enemies
        enemySpawnTimer += timeStep;
        if (enemySpawnTimer > ENEMY_SPAWN_RATE)
        {
            enemySpawnTimer = 0;
            i32 numEnemies = 0;
            Vector<Node*> ninjaNodes = gameScene->GetChildrenWithComponent("Ninja", true);
            for (i32 i = 0; i < ninjaNodes.Size(); ++i)
            {
                Ninja* ninja = ninjaNodes[i]->GetComponent<Ninja>();
                if (ninja->side == SIDE_ENEMY)
                    ++numEnemies;
            }

            if (numEnemies < maxEnemies)
            {
                const float maxOffset = 40.f;
                float offset = Random(maxOffset * 2.0f) - maxOffset;
                // Random north/east/south/west direction
                i32 dir = Rand() & 3;
                dir *= 90;
                Quaternion rotation(0.f, (float)dir, 0.f);

                Node* enemyNode = SpawnObject(gameScene, rotation * Vector3(offset, 10.f, -120.f),
rotation, "ninja");

                // Initialize variables
                Ninja* enemyNinja = enemyNode->GetComponent<Ninja>();
                enemyNinja->side = SIDE_ENEMY;
                enemyNinja->controller.reset(new AIController());
                RigidBody* enemyBody = enemyNode->GetComponent<RigidBody>();
                enemyBody->SetLinearVelocity(rotation * Vector3(0.f, 10.f, 30.f));
            }
        }
    }

    void CheckEndAndRestart()
    {
        // Only check end of game if singleplayer or client
        if (runServer)
            return;

        // Check if player node has vanished
        Node* playerNode = FindOwnNode();
        if (gameOn && !playerNode)
        {
            gameOn = false;
            SetMessage("Press Fire or Jump to restart!");
            return;
        }

        // Check for restart (singleplayer only)
        if (!gameOn && singlePlayer && playerControls.IsPressed(CTRL_FIRE | CTRL_JUMP,
prevPlayerControls))
```

```
            StartGame(nullptr);
    }

    void UpdateControls()
    {
        Input* input = GetSubsystem<Input>();
        Graphics* graphics = GetSubsystem<Graphics>();
        Console* console = GetSubsystem<Console>();
        Network* network = GetSubsystem<Network>();

        if (singlePlayer || runClient)
        {
            prevPlayerControls = playerControls;
            playerControls.Set(CTRL_ALL, false);

            if (touchEnabled)
            {
                for (i32 i = 0; i < input->GetNumTouches(); ++i)
                {
                    TouchState* touch = input->GetTouch(i);
                    if (!touch->touchedElement_)
                    {
                        // Touch on empty space
                        playerControls.yaw_ += TOUCH_SENSITIVITY * gameCamera->GetFov() /
graphics->GetHeight() * touch->delta_.x_;
                        playerControls.pitch_ += TOUCH_SENSITIVITY * gameCamera->GetFov() /
graphics->GetHeight() * touch->delta_.y_;
                    }
                }
            }

            if (input->GetNumJoysticks() > 0)
            {
                JoystickState* joystick = touchEnabled ? input->GetJoystick(screenJoystickID) :
input->GetJoystickByIndex(0);
                if (joystick->GetNumButtons() > 0)
                {
                    if (joystick->GetButtonDown(0))
                        playerControls.Set(CTRL_JUMP, true);
                    if (joystick->GetButtonDown(1))
                        playerControls.Set(CTRL_FIRE, true);
                    if (joystick->GetNumButtons() >= 6)
                    {
                        if (joystick->GetButtonDown(4))
                            playerControls.Set(CTRL_JUMP, true);
                        if (joystick->GetButtonDown(5))
                            playerControls.Set(CTRL_FIRE, true);
                    }
                    if (joystick->GetNumHats() > 0)
                    {
                        if ((joystick->GetHatPosition(0) & HAT_LEFT) != 0)
                            playerControls.Set(CTRL_LEFT, true);
                        if ((joystick->GetHatPosition(0) & HAT_RIGHT) != 0)
                            playerControls.Set(CTRL_RIGHT, true);
                        if ((joystick->GetHatPosition(0) & HAT_UP) != 0)
                            playerControls.Set(CTRL_UP, true);
                        if ((joystick->GetHatPosition(0) & HAT_DOWN) != 0)
                            playerControls.Set(CTRL_DOWN, true);
                    }
                    if (joystick->GetNumAxes() >= 2)
                    {
                        if (joystick->GetAxisPosition(0) < -JOY_MOVE_DEAD_ZONE)
                            playerControls.Set(CTRL_LEFT, true);
                        if (joystick->GetAxisPosition(0) > JOY_MOVE_DEAD_ZONE)
                            playerControls.Set(CTRL_RIGHT, true);
                        if (joystick->GetAxisPosition(1) < -JOY_MOVE_DEAD_ZONE)
                            playerControls.Set(CTRL_UP, true);
                        if (joystick->GetAxisPosition(1) > JOY_MOVE_DEAD_ZONE)
                            playerControls.Set(CTRL_DOWN, true);
```

```
                    }
                    if (joystick->GetNumAxes() >= 4)
                    {
                        float lookX = joystick->GetAxisPosition(2);
                        float lookY = joystick->GetAxisPosition(3);

                        if (lookX < -JOY_LOOK_DEAD_ZONE)
                            playerControls.yaw_ -= JOY_SENSITIVITY * lookX * lookX;
                        if (lookX > JOY_LOOK_DEAD_ZONE)
                            playerControls.yaw_ += JOY_SENSITIVITY * lookX * lookX;
                        if (lookY < -JOY_LOOK_DEAD_ZONE)
                            playerControls.pitch_ -= JOY_SENSITIVITY * lookY * lookY;
                        if (lookY > JOY_LOOK_DEAD_ZONE)
                            playerControls.pitch_ += JOY_SENSITIVITY * lookY * lookY;
                    }
                }
            }

            // For the triggered actions (fire & jump) check also for press, in case the FPS is
low
            // and the key was already released
            if (!console || !console->IsVisible())
            {
                if (input->GetKeyDown(KEY_W))
                    playerControls.Set(CTRL_UP, true);
                if (input->GetKeyDown(KEY_S))
                    playerControls.Set(CTRL_DOWN, true);
                if (input->GetKeyDown(KEY_A))
                    playerControls.Set(CTRL_LEFT, true);
                if (input->GetKeyDown(KEY_D))
                    playerControls.Set(CTRL_RIGHT, true);
                if (input->GetKeyDown(KEY_LCTRL) || input->GetKeyPress(KEY_LCTRL))
                    playerControls.Set(CTRL_FIRE, true);
                if (input->GetKeyDown(KEY_SPACE) || input->GetKeyPress(KEY_SPACE))
                    playerControls.Set(CTRL_JUMP, true);

                if (input->GetMouseButtonDown(MOUSEB_LEFT) || input-
>GetMouseButtonPress(MOUSEB_LEFT))
                    playerControls.Set(CTRL_FIRE, true);
                if (input->GetMouseButtonDown(MOUSEB_RIGHT) || input-
>GetMouseButtonPress(MOUSEB_RIGHT))
                    playerControls.Set(CTRL_JUMP, true);

                playerControls.yaw_ += MOUSE_SENSITIVITY * input->GetMouseMoveX();
                playerControls.pitch_ += MOUSE_SENSITIVITY * input->GetMouseMoveY();
                playerControls.pitch_ = Clamp(playerControls.pitch_, -60.0f, 60.0f);
            }

            // In singleplayer, set controls directly on the player's ninja. In multiplayer,
transmit to server
            if (singlePlayer)
            {
                Node* playerNode = gameScene->GetChild("Player", true);
                if (playerNode)
                {
                    Ninja* playerNinja = playerNode->GetComponent<Ninja>();
                    playerNinja->controls = playerControls;
                }
            }
            else if (network->GetServerConnection() != nullptr)
            {
                // Set the latest yaw & pitch to server controls, and accumulate the buttons so
that we do not miss any presses
                network->GetServerConnection()->controls_.yaw_ = playerControls.yaw_;
                network->GetServerConnection()->controls_.pitch_ = playerControls.pitch_;
                network->GetServerConnection()->controls_.buttons_ |= playerControls.buttons_;

                // Tell the camera position to server for interest management
                network->GetServerConnection()->SetPosition(gameCameraNode->GetWorldPosition());
```

```
            }
        }

        if (runServer)
        {
            // Apply each connection's controls to the ninja they control
            for (i32 i = 0; i < players.Size(); ++i)
            {
                Node* playerNode = FindPlayerNode(i);
                if (playerNode)
                {
                    Ninja* playerNinja = playerNode->GetComponent<Ninja>();
                    playerNinja->controls = players[i].connection->controls_;
                }
                else
                {
                    // If player has no ninja, respawn if fire/jump is pressed
                    if (players[i].connection->controls_.IsPressed(CTRL_FIRE | CTRL_JUMP,
players[i].lastControls))
                        SpawnPlayer(players[i].connection);
                }
                players[i].lastControls = players[i].connection->controls_;
            }
        }
    }

    void UpdateCamera()
    {
        if (GetSubsystem<Engine>()->IsHeadless())
            return;

        // On the server, use a simple freelook camera
        if (runServer)
        {
            UpdateFreelookCamera();
            return;
        }

        Node* playerNode = FindOwnNode();
        if (!playerNode)
            return;

        Vector3 pos = playerNode->GetPosition();
        Quaternion dir;

        // Make controls seem more immediate by forcing the current mouse yaw to player ninja's Y-
axis rotation
        if (playerNode->GetVar("Health").GetI32() > 0)
            playerNode->SetRotation(Quaternion(0.f, playerControls.yaw_, 0.f));

        dir = dir * Quaternion(playerNode->GetRotation().YawAngle(), Vector3(0.f, 1.f, 0.f));
        dir = dir * Quaternion(playerControls.pitch_, Vector3(1.f, 0.f, 0.f));

        Vector3 aimPoint = pos + Vector3(0.f, 1.f, 0.f);
        Vector3 minDist = aimPoint + dir * Vector3(0.f, 0.f, -CAMERA_MIN_DIST);
        Vector3 maxDist = aimPoint + dir * Vector3(0.f, 0.f, -CAMERA_MAX_DIST);

        // Collide camera ray with static objects (collision mask 2)
        Vector3 rayDir = (maxDist - minDist).Normalized();
        float rayDistance = CAMERA_MAX_DIST - CAMERA_MIN_DIST + CAMERA_SAFETY_DIST;
        PhysicsRaycastResult result;
        gameScene->GetComponent<PhysicsWorld>()->RaycastSingle(result, Ray(minDist, rayDir),
rayDistance, 2);
        if (result.body_)
            rayDistance = Min(rayDistance, result.distance_ - CAMERA_SAFETY_DIST);

        gameCameraNode->SetPosition(minDist + rayDir * rayDistance);
        gameCameraNode->SetRotation(dir);
    }
```

```
    void UpdateFreelookCamera()
    {
        Console* console = GetSubsystem<Console>();
        Time* time = GetSubsystem<Time>();
        Input* input = GetSubsystem<Input>();

        if (!console || !console->IsVisible())
        {
            float timeStep = time->GetTimeStep();
            float speedMultiplier = 1.0f;
            if (input->GetKeyDown(KEY_LSHIFT))
                speedMultiplier = 5.0f;
            if (input->GetKeyDown(KEY_LCTRL))
                speedMultiplier = 0.1f;

            if (input->GetKeyDown(KEY_W))
                gameCameraNode->Translate(Vector3(0.f, 0.f, 10.f) * timeStep * speedMultiplier);
            if (input->GetKeyDown(KEY_S))
                gameCameraNode->Translate(Vector3(0.f, 0.f, -10.f) * timeStep * speedMultiplier);
            if (input->GetKeyDown(KEY_A))
                gameCameraNode->Translate(Vector3(-10.f, 0.f, 0.f) * timeStep * speedMultiplier);
            if (input->GetKeyDown(KEY_D))
                gameCameraNode->Translate(Vector3(10.f, 0.f, 0.f) * timeStep * speedMultiplier);

            playerControls.yaw_ += MOUSE_SENSITIVITY * input->GetMouseMoveX();
            playerControls.pitch_ += MOUSE_SENSITIVITY * input->GetMouseMoveY();
            playerControls.pitch_ = Clamp(playerControls.pitch_, -90.0f, 90.0f);
            gameCameraNode->SetRotation(Quaternion(playerControls.pitch_, playerControls.yaw_,
0.f));
        }
    }

    void UpdateStatus()
    {
        Engine* engine = GetSubsystem<Engine>();

        if (engine->IsHeadless() || runServer)
            return;

        if (singlePlayer)
        {
            if (players.Size() > 0)
                scoreText->SetText("Score " + String(players[0].score));
            if (hiscores.Size() > 0)
                hiscoreText->SetText("Hiscore " + String(hiscores[0].score));
        }

        Node* playerNode = FindOwnNode();
        if (playerNode)
        {
            i32 health = 0;
            if (singlePlayer)
            {
                GameObject* object = playerNode->GetDerivedComponent<GameObject>();
                health = object->health;
            }
            else
            {
                // In multiplayer the client does not have script logic components, but health is
replicated via node user variables
                health = playerNode->GetVar("Health").GetI32();
            }
            healthBar->SetWidth(116 * health / PLAYER_HEALTH);
        }
    }

    void HandleScreenMode(StringHash eventType, VariantMap& eventData)
    {
```

```
        Graphics* graphics = GetSubsystem<Graphics>();
        i32 height = graphics->GetHeight() / 22;
        if (height > 64)
            height = 64;
        sight->SetSize(height, height);
        messageText->SetPosition(0, -height * 2);
    }
};

} // namespace Urho3D

URHO3D_DEFINE_APPLICATION_MAIN(App);

================================================================================
ninja.cpp
----------
// Copyright (c) 2022-2023 the Dviglo project
// Copyright (c) 2008-2023 the Urho3D project
// License: MIT

#include "ninja.h"

#include "snowball.h"
#include "utilities/spawn.h"

namespace Urho3D
{

static constexpr i32 LAYER_MOVE = 0;
static constexpr i32 LAYER_ATTACK = 1;

static constexpr float NINJA_MOVE_FORCE = 25.f;
static constexpr float NINJA_AIR_MOVE_FORCE = 1.f;
static constexpr float NINJA_DAMPING_FORCE = 5.f;
static constexpr float NINJA_JUMP_FORCE = 450.f;
static const Vector3 NINJA_THROW_VELOCITY(0.f, 4.25f, 20.f);
static const Vector3 NINJA_THROW_POSITION(0.f, 0.2f, 1.f);
static constexpr float NINJA_THROW_DELAY = 0.1f;
static constexpr float NINJA_CORPSE_DURATION = 3.f;
static constexpr i32 NINJA_POINTS = 250;

void Ninja::RegisterObject(Context* context)
{
    context->RegisterFactory<Ninja>();
}

Ninja::Ninja(Context* context)
        : GameObject(context)
    , okToJump(false)
    , smoke(false)
    , inAirTime(1.f)
    , onGroundTime(0.f)
    , throwTime(0.f)
    , deathTime(0.f)
    , deathDir(0.f)
    , dirChangeTime(0.f)
    , aimX(0.f)
    , aimY(0.f)
{
    health = maxHealth = 2;
    onGround = false;
    isSliding = false;
}

void Ninja::DelayedStart()
{
    SubscribeToEvent(node_, E_NODECOLLISION, URHO3D_HANDLER(Ninja, HandleNodeCollision));

    // Get horizontal aim from initial rotation
```

```
    aimX = controls.yaw_ = node_->GetRotation().YawAngle();

    // Start playing the idle animation immediately, even before the first physics update
    AnimationController* animCtrl = node_->GetChild(0)->GetComponent<AnimationController>();
    animCtrl->PlayExclusive("Models/NinjaSnowWar/Ninja_Idle3.ani", LAYER_MOVE, true);
}

void Ninja::SetControls(const Controls& newControls)
{
    controls = newControls;
}

Quaternion Ninja::GetAim()
{
    Quaternion q = Quaternion(aimX, Vector3(0.f, 1.f, 0.f));
    q = q * Quaternion(aimY, Vector3(1.f, 0.f, 0.f));
    return q;
}

void Ninja::FixedUpdate(float timeStep)
{
    // For multiplayer, replicate the health into the node user variables
    node_->SetVar("Health", health);

    if (health <= 0)
    {
        DeathUpdate(timeStep);
        return;
    }

    // AI control if controller exists
    if (controller)
        controller->Control(this, node_, timeStep);

    RigidBody* body = node_->GetComponent<RigidBody>();
    AnimationController* animCtrl = node_->GetChild(0)->GetComponent<AnimationController>();

    // Turning / horizontal aiming
    if (aimX != controls.yaw_)
        aimX = controls.yaw_;

    // Vertical aiming
    if (aimY != controls.pitch_)
        aimY = controls.pitch_;

    // Force the physics rotation
    Quaternion q(aimX, Vector3(0.f, 1.f, 0.f));
    body->SetRotation(q);

    // Movement ground/air
    Vector3 vel = body->GetLinearVelocity();
    if (onGround)
    {
        // If landed, play a particle effect at feet (use the AnimatedModel node)
        if (inAirTime > 0.5)
            SpawnParticleEffect(node_->GetScene(), node_->GetChild(0)->GetWorldPosition(),
"Particle/SnowExplosion.xml", 1);

        inAirTime = 0;
        onGroundTime += timeStep;
    }
    else
    {
        onGroundTime = 0;
        inAirTime += timeStep;
    }

    if (inAirTime < 0.3f && !isSliding)
    {
```

```
        bool sideMove = false;

        // Movement in four directions
        if (controls.IsDown(CTRL_UP | CTRL_DOWN | CTRL_LEFT | CTRL_RIGHT))
        {
            float animDir = 1.0f;
            Vector3 force(0, 0, 0);
            if (controls.IsDown(CTRL_UP))
                force += q * Vector3(0, 0, 1);
            if (controls.IsDown(CTRL_DOWN))
            {
                animDir = -1.0f;
                force += q * Vector3(0, 0, -1);
            }
            if (controls.IsDown(CTRL_LEFT))
            {
                sideMove = true;
                force += q * Vector3(-1, 0, 0);
            }
            if (controls.IsDown(CTRL_RIGHT))
            {
                sideMove = true;
                force += q * Vector3(1, 0, 0);
            }
            // Normalize so that diagonal strafing isn't faster
            force.Normalize();
            force *= NINJA_MOVE_FORCE;
            body->ApplyImpulse(force);

            // Walk or sidestep animation
            if (sideMove)
            {
                animCtrl->PlayExclusive("Models/NinjaSnowWar/Ninja_Stealth.ani", LAYER_MOVE, true,
0.2f);
                animCtrl->SetSpeed("Models/NinjaSnowWar/Ninja_Stealth.ani", animDir * 2.2f);
            }
            else
            {
                animCtrl->PlayExclusive("Models/NinjaSnowWar/Ninja_Walk.ani", LAYER_MOVE, true,
0.2f);
                animCtrl->SetSpeed("Models/NinjaSnowWar/Ninja_Walk.ani", animDir * 1.6f);
            }
        }
        else
        {
            // Idle animation
            animCtrl->PlayExclusive("Models/NinjaSnowWar/Ninja_Idle3.ani", LAYER_MOVE, true,
0.2f);
        }

        // Overall damping to cap maximum speed
        body->ApplyImpulse(Vector3(-NINJA_DAMPING_FORCE * vel.x_, 0, -NINJA_DAMPING_FORCE *
vel.z_));

        // Jumping
        if (controls.IsDown(CTRL_JUMP))
        {
            if (okToJump && inAirTime < 0.1f)
            {
                // Lift slightly off the ground for better animation
                body->SetPosition(body->GetPosition() + Vector3(0.f, 0.03f, 0.f));
                body->ApplyImpulse(Vector3(0.f, NINJA_JUMP_FORCE, 0.f));
                inAirTime = 1.0f;
                animCtrl->PlayExclusive("Models/NinjaSnowWar/Ninja_JumpNoHeight.ani", LAYER_MOVE,
false, 0.1f);
                animCtrl->SetTime("Models/NinjaSnowWar/Ninja_JumpNoHeight.ani", 0.0f); // Always
play from beginning
                okToJump = false;
            }
```

```
        }
        else okToJump = true;
    }
    else
    {
        // Motion in the air
        // Note: when sliding a steep slope, control (or damping) isn't allowed!
        if (inAirTime > 0.3f && !isSliding)
        {
            if (controls.IsDown(CTRL_UP | CTRL_DOWN | CTRL_LEFT | CTRL_RIGHT))
            {
                Vector3 force(0, 0, 0);
                if (controls.IsDown(CTRL_UP))
                    force += q * Vector3(0, 0, 1);
                if (controls.IsDown(CTRL_DOWN))
                    force += q * Vector3(0, 0, -1);
                if (controls.IsDown(CTRL_LEFT))
                    force += q * Vector3(-1, 0, 0);
                if (controls.IsDown(CTRL_RIGHT))
                    force += q * Vector3(1, 0, 0);
                // Normalize so that diagonal strafing isn't faster
                force.Normalize();
                force *= NINJA_AIR_MOVE_FORCE;
                body->ApplyImpulse(force);
            }
        }

        // Falling/jumping/sliding animation
        if (inAirTime > 0.1f)
            animCtrl->PlayExclusive("Models/NinjaSnowWar/Ninja_JumpNoHeight.ani", LAYER_MOVE,
false, 0.1f);
    }

    // Shooting
    if (throwTime >= 0)
        throwTime -= timeStep;

    // Start fading the attack animation after it has progressed past a certain point
    if (animCtrl->GetTime("Models/NinjaSnowWar/Ninja_Attack1.ani") > 0.1f)
        animCtrl->Fade("Models/NinjaSnowWar/Ninja_Attack1.ani", 0.0f, 0.5f);

    if (controls.IsPressed(CTRL_FIRE, prevControls) && throwTime <= 0.f)
    {
        Vector3 projectileVel = GetAim() * NINJA_THROW_VELOCITY;

        animCtrl->Play("Models/NinjaSnowWar/Ninja_Attack1.ani", LAYER_ATTACK, false, 0.0f);
        animCtrl->SetTime("Models/NinjaSnowWar/Ninja_Attack1.ani", 0.0f); // Always play from
beginning

        Node* snowball = SpawnObject(node_->GetScene(), node_->GetPosition() + vel * timeStep + q
* NINJA_THROW_POSITION, GetAim(), "snowball");
        RigidBody* snowballBody = snowball->GetComponent<RigidBody>();
        snowballBody->SetLinearVelocity(projectileVel);
        Snowball* snowballObject = snowball->GetComponent<Snowball>();
        snowballObject->side = side;
        snowballObject->creatorID = node_->GetID();

        PlaySound("Sounds/NutThrow.wav");

        throwTime = NINJA_THROW_DELAY;
    }

    prevControls = controls;

    ResetWorldCollision();
}

void Ninja::DeathUpdate(float timeStep)
{
```

```
    RigidBody* body = node_->GetComponent<RigidBody>();
    CollisionShape* shape = node_->GetComponent<CollisionShape>();
    Node* modelNode = node_->GetChild(0);
    AnimationController* animCtrl = modelNode->GetComponent<AnimationController>();
    AnimatedModel* model = modelNode->GetComponent<AnimatedModel>();

    Vector3 vel = body->GetLinearVelocity();

    // Overall damping to cap maximum speed
    body->ApplyImpulse(Vector3(-NINJA_DAMPING_FORCE * vel.x_, 0, -NINJA_DAMPING_FORCE * vel.z_));

    // Collide only to world geometry
    body->SetCollisionMask(2);

    // Pick death animation on first death update
    if (deathDir == 0)
    {
        if (Random(1.0f) < 0.5f)
            deathDir = -1.f;
        else
            deathDir = 1.f;

        PlaySound("Sounds/SmallExplosion.wav");

        VariantMap eventData;
        eventData["Points"] = NINJA_POINTS;
        eventData["Receiver"] = lastDamageCreatorID;
        eventData["DamageSide"] = lastDamageSide;
        SendEvent("Points", eventData);
        SendEvent("Kill", eventData);
    }

    deathTime += timeStep;

    // Move the model node to center the corpse mostly within the physics cylinder
    // (because of the animation)
    if (deathDir < 0.f)
    {
        // Backward death
        animCtrl->StopLayer(LAYER_ATTACK, 0.1f);
        animCtrl->PlayExclusive("Models/NinjaSnowWar/Ninja_Death1.ani", LAYER_MOVE, false, 0.2f);
        animCtrl->SetSpeed("Models/NinjaSnowWar/Ninja_Death1.ani", 0.5f);
        if (deathTime >= 0.3f && deathTime < 0.8f)
            modelNode->Translate(Vector3(0.f, 0.f, 4.25f * timeStep));
    }
    else if (deathDir > 0.f)
    {
        // Forward death
        animCtrl->StopLayer(LAYER_ATTACK, 0.1f);
        animCtrl->PlayExclusive("Models/NinjaSnowWar/Ninja_Death2.ani", LAYER_MOVE, false, 0.2f);
        animCtrl->SetSpeed("Models/NinjaSnowWar/Ninja_Death2.ani", 0.5f);
        if (deathTime >= 0.4f && deathTime < 0.8f)
            modelNode->Translate(Vector3(0.f, 0.f, -4.25f * timeStep));
    }

    // Create smokecloud just before vanishing
    if (deathTime > NINJA_CORPSE_DURATION - 1.f && !smoke)
    {
        SpawnParticleEffect(node_->GetScene(), node_->GetPosition() + Vector3(0.f, -0.4f, 0.f),
"Particle/Smoke.xml", 8.f);
        smoke = true;
    }

    if (deathTime > NINJA_CORPSE_DURATION)
    {
        SpawnObject(node_->GetScene(), node_->GetPosition() + Vector3(0.f, -0.5f, 0.f),
Quaternion(), "light_flash");
        SpawnSound(node_->GetScene(), node_->GetPosition() + Vector3(0.f, -0.5f, 0.f),
"Sounds/BigExplosion.wav", 2.f);
```

```
        node_->Remove();
    }
}

bool Ninja::Heal(i32 amount)
{
    if (health == maxHealth)
        return false;

    health += amount;
    if (health > maxHealth)
        health = maxHealth;
    // If player, play the "powerup" sound
    if (side == SIDE_PLAYER)
        PlaySound("Sounds/Powerup.wav");

    return true;
}

} // namespace Urho3D


================================================================================
ninja.h
---------
// Copyright (c) 2022-2023 the Dviglo project
// Copyright (c) 2008-2023 the Urho3D project
// License: MIT

#pragma once

#include <memory>

#include "game_object.h"

#include "ai_controller.h"

namespace Urho3D
{

class Ninja : public GameObject
{
    URHO3D_OBJECT(Ninja, GameObject);

public:
    static void RegisterObject(Context* context);

    Controls controls;
    Controls prevControls;
    std::unique_ptr<AIController> controller;
    bool okToJump;
    bool smoke;
    float inAirTime;
    float onGroundTime;
    float throwTime;
    float deathTime;
    float deathDir;
    float dirChangeTime;
    float aimX;
    float aimY;

    Ninja(Context* context);
    void DelayedStart() override;
    void SetControls(const Controls& newControls);
    Quaternion GetAim();
    void FixedUpdate(float timeStep) override;
    void DeathUpdate(float timeStep);
    bool Heal(i32 amount) override;
};
```

```
} // namespace Urho3D

================================================================================
player.cpp
------------
// Copyright (c) 2022-2023 the Dviglo project
// Copyright (c) 2008-2023 the Urho3D project
// License: MIT

#include "player.h"

namespace Urho3D
{

Player::Player()
    : score(0)
    , nodeID(0)
{
}

} // namespace Urho3D
================================================================================
player.h
---------
// Copyright (c) 2022-2023 the Dviglo project
// Copyright (c) 2008-2023 the Urho3D project
// License: MIT

#pragma once

#include <Urho3D/Urho3DAll.h>

namespace Urho3D
{

struct Player
{
    i32 score;
    String name;
    NodeId nodeID;
    WeakPtr<Connection> connection;
    Controls lastControls;

    Player();
};

struct HiscoreEntry
{
    i32 score;
    String name;
};

} // namespace Urho3D


================================================================================
potion.cpp
-----------
// Copyright (c) 2022-2023 the Dviglo project
// Copyright (c) 2008-2023 the Urho3D project
// License: MIT

#include "potion.h"

namespace Urho3D
{

static constexpr int POTION_HEAL_AMOUNT = 5;
```

```
void Potion::RegisterObject(Context* context)
{
    context->RegisterFactory<Potion>();
}

Potion::Potion(Context* context)
    : GameObject(context)
{
    healAmount = POTION_HEAL_AMOUNT;
}

void Potion::Start()
{
    SubscribeToEvent(node_, E_NODECOLLISION, URHO3D_HANDLER(Potion, HandleNodeCollision));
}

void Potion::ObjectCollision(GameObject& otherObject, VariantMap& eventData)
{
    if (healAmount > 0)
    {
        if (otherObject.Heal(healAmount))
        {
            // Could also remove the potion directly, but this way it gets removed on next update
            healAmount = 0;
            duration = 0;
        }
    }
}

} // namespace Urho3D

================================================================================
potion.h
---------
// Copyright (c) 2022-2023 the Dviglo project
// Copyright (c) 2008-2023 the Urho3D project
// License: MIT

#pragma once

#include "game_object.h"

namespace Urho3D
{

class Potion : public GameObject
{
    URHO3D_OBJECT(Potion, GameObject);

private:
    i32 healAmount;

public:
    static void RegisterObject(Context* context);

    Potion(Context* context);
    void Start() override;
    void ObjectCollision(GameObject& otherObject, VariantMap& eventData) override;
};

} // namespace Urho3D

================================================================================
snow_create.cpp
-------------
// Copyright (c) 2022-2023 the Dviglo project
// Copyright (c) 2008-2023 the Urho3D project
// License: MIT
```

```
#include "snow_crate.h"
#include "utilities/spawn.h"

namespace Urho3D
{

static constexpr i32 SNOWCRATE_HEALTH = 5;
static constexpr i32 SNOWCRATE_POINTS = 250;

void SnowCrate::RegisterObject(Context* context)
{
    context->RegisterFactory<SnowCrate>();
}

SnowCrate::SnowCrate(Context* context)
    : GameObject(context)
{
    health = maxHealth = SNOWCRATE_HEALTH;
}

void SnowCrate::Start()
{
    SubscribeToEvent(node_, E_NODECOLLISION, URHO3D_HANDLER(SnowCrate, HandleNodeCollision));
}

void SnowCrate::FixedUpdate(float timeStep)
{
    if (health <= 0)
    {
        SpawnParticleEffect(node_->GetScene(), node_->GetPosition(),
"Particle/SnowExplosionBig.xml", 2);
        SpawnObject(node_->GetScene(), node_->GetPosition(), Quaternion(), "potion");

        VariantMap eventData;
        eventData["Points"] = SNOWCRATE_POINTS;
        eventData["Receiver"] = lastDamageCreatorID;
        eventData["DamageSide"] = lastDamageSide;
        SendEvent("Points", eventData);

        node_->Remove();
    }
}

} // namespace Urho3D

================================================================================
snow_create.h
-------------
// Copyright (c) 2022-2023 the Dviglo project
// Copyright (c) 2008-2023 the Urho3D project
// License: MIT

#pragma once

#include "game_object.h"

namespace Urho3D
{

class SnowCrate : public GameObject
{
    URHO3D_OBJECT(SnowCrate, GameObject);

public:
    static void RegisterObject(Context* context);

    SnowCrate(Context* context);
```

```
    void Start() override;
    void FixedUpdate(float timeStep) override;
};

} // namespace Urho3D

================================================================================
snowball.cpp
------------
// Copyright (c) 2022-2023 the Dviglo project
// Copyright (c) 2008-2023 the Urho3D project
// License: MIT

#include "snowball.h"
#include "utilities/spawn.h"

namespace Urho3D
{

static constexpr float SNOWBALL_MIN_HIT_SPEED = 1.f;
static constexpr float SNOWBALL_DAMPING_FORCE = 20.f;
static constexpr float SNOWBALL_DURATION = 5.f;
static constexpr float SNOWBALL_GROUND_HIT_DURATION = 1.f;
static constexpr float SNOWBALL_OBJECT_HIT_DURATION = 0.f;
static constexpr i32 SNOWBALL_DAMAGE = 1;

void Snowball::RegisterObject(Context* context)
{
    context->RegisterFactory<Snowball>();
}

Snowball::Snowball(Context* context)
    : GameObject(context)
{
    duration = SNOWBALL_DURATION;
    hitDamage = SNOWBALL_DAMAGE;
}

void Snowball::Start()
{
    SubscribeToEvent(node_, E_NODECOLLISION, URHO3D_HANDLER(Snowball, HandleNodeCollision));
}

void Snowball::FixedUpdate(float timeStep)
{
    // Apply damping when rolling on the ground, or near disappearing
    RigidBody* body = node_->GetComponent<RigidBody>();

    if (onGround || duration < SNOWBALL_GROUND_HIT_DURATION)
    {
        Vector3 vel = body->GetLinearVelocity();
        body->ApplyForce(Vector3(-SNOWBALL_DAMPING_FORCE * vel.x_, 0.f, -SNOWBALL_DAMPING_FORCE *
vel.z_));
    }

    // Disappear when duration expired
    if (duration >= 0.f)
    {
        duration -= timeStep;
        if (duration <= 0.f)
        {
            SpawnParticleEffect(node_->GetScene(), node_->GetPosition(),
"Particle/SnowExplosion.xml", 1);
            node_->Remove();
        }
    }
}

void Snowball::WorldCollision(VariantMap& eventData)
```

```
{
    GameObject::WorldCollision(eventData);

    // If hit the ground, disappear after a short while
    if (duration > SNOWBALL_GROUND_HIT_DURATION)
        duration = SNOWBALL_GROUND_HIT_DURATION;
}

void Snowball::ObjectCollision(GameObject& otherObject, VariantMap& eventData)
{
    if (hitDamage > 0)
    {
        RigidBody* body = node_->GetComponent<RigidBody>();

        if (body->GetLinearVelocity().Length() >= SNOWBALL_MIN_HIT_SPEED)
        {
            if (side != otherObject.side)
            {
                otherObject.Damage(*this, hitDamage);
                // Create a temporary node for the hit sound
                SpawnSound(node_->GetScene(), node_->GetPosition(), "Sounds/PlayerFistHit.wav",
0.2f);
            }

            hitDamage = 0;
        }
    }
    if (duration > SNOWBALL_OBJECT_HIT_DURATION)
        duration = SNOWBALL_OBJECT_HIT_DURATION;
}

} // namespace Urho3D

======================================================================
snowball.h
------------
// Copyright (c) 2022-2023 the Dviglo project
// Copyright (c) 2008-2023 the Urho3D project
// License: MIT

#pragma once

#include "game_object.h"

namespace Urho3D
{

class Snowball : public GameObject
{
    URHO3D_OBJECT(Snowball, GameObject);

private:
    int hitDamage;

public:
    static void RegisterObject(Context* context);

    Snowball(Context* context);

    void Start() override;
    void FixedUpdate(float timeStep) override;
    void WorldCollision(VariantMap& eventData) override;
    void ObjectCollision(GameObject& otherObject, VariantMap& eventData) override;
};

} // namespace Urho3D
```